

ERABI

えらび

ローカル判断エンジン 総合設計書

自然文と選択肢から、確率分布を返す。

小さく実装し、正しく測り、必要な機能だけを育てる。

FOR CODEX / GOOGLE ANTIGRAVITY

技術背景・モデル設計・データ・学習・校正・評価・テスト・運用

最初の実装範囲：M0 + M1

環境確認と既存モデルの実推論・比較から開始。

大規模学習、課金API、UI、分散基盤はまだ作らない。

VERSION 1.0 / 2026-09-18

実装前の設計資料。学習済みモデル・実機ベンチマークは含まない。

Contents

設計の正本は同梱の ERABI_DESIGN.md。実装エージェントはMarkdownを参照してください。

0. この文書の使い方	3
1. 名前・背景・開発の前提	3
2. 目的・非目的・完成の定義	4
3. 技術背景と判断を誤りやすい点	5
4. 初期の設計判断	6
5. 全体の処理フロー	6
6. 入出力仕様	7
7. 第一ルート：GLiClassを使う	8
8. 第二ルート：日本語エンコーダーから判断モデルを作る	9
9. データ設計と分割	10
10. 教師データの作成と品質管理	11
11. 学習設計	12
12. 校正・保留・自動処理	14
13. 評価設計	15
14. 過剰にしないテスト方針	16
15. ローカルAPIと安全な運用	17
16. コード構成と依存関係	18
17. 環境確認とセットアップの手順	19
18. 速度・VRAM・高速化の検証	19
19. 実装フェーズと完了条件	20
20. Codex / Antigravityへの引き渡し	21
21. 主な失敗モードと対処	22
22. 将来拡張の設計メモ	22
23. 完成前のチェックリスト	22
24. 参考資料と根拠	23

ERABI（えらび）総合設計書

自然文・判断基準・可変の選択肢から、確率分布を返すローカル判断エンジン

文書版：1.0 / 作成・公開情報確認日：2026-09-18（日本時間）

対象：手元のCodexまたはGoogle Antigravityで実装する、個人開発・実験用プロジェクト

状態：実装前の設計。性能、VRAM使用量、学習時間、精度改善は未実測。

最初に作るのは「任意の選択肢を採点できる、小さなローカルモデルと評価系」。Jevの非公開モデルやRLCDを再現できたとは主張しない。まず単一選択を完成させ、必要な機能だけを増やす。

0. この文書の使い方

実装エージェントは、最初にルートのAGENTS.md、次にKICKOFF.mdを読む。本書は技術・仕様の正本であり、PDFは閲覧用の同内容版である。すべての将来機能を一度に実装してはならない。

初回はM0（環境確認）とM1（既存モデルによる実測）までを進める。必要な学習データやGPUがなければ、架空の結果で埋めず、実行できた部分と未実行部分を区別する。以後は実験結果に応じてM2以降へ進む。

本書の数字には三種類がある。入力上限やAPI形式は**初期仕様**、データ量・学習率・採否目安は**実験の出発点**、モデルカード等の記述は**公開資料で確認した情報**である。実験案を実測結果や必要最低量と取り違えない。

読む順番

読者・目的	優先して読む節
プロジェクト全体を理解する	1～5、19、23
最初の推論を実装する	6～7、14、16～17
追加学習を実装する	8～11、13～14
確率を実運用に使う	12～15、18、21
エージェントの作業範囲を決める	19～20、AGENTS.md、KICKOFF.md

1. 名前・背景・開発の前提

1.1 プロジェクト名

ERABI（えらび）を採用案とする。「選び、判断する」という目的が名前から伝わり、文章生成モデルと区別しやすい。リポジトリ名はerabi-local、Pythonのインポート名はerabi、表示名はERABIとする。略語の後付けの展開は設けない。

これは個人開発用の仮称であり、商標・ドメイン・パッケージ名の独占性は確認していない。公開配布・登録前に名称の重複を確認する。JevやTypeSafeの公式版・互換認定版であるような名称や表示は使わない。

1.2 これまでの検討を引き継ぐ

狙いは、LLMへ毎回文章を書かせなくてもよい処理をローカル化すること。入力は「状況を表す自然文」「判断基準を含む質問」「その場で指定する候補」。出力は候補ごとの数値で、JSON自体はプログラムが組み立てる。

ユーザーの発想は、自然文・候補・正解または教師確率を一組として学習する方式である。この方針を主軸とする。最初から事前学習をやり直したり、強化学習基盤を作ったりしない。

JevとGLiClassの派生関係は未確認。ERABIはGLiClass等の公開技術を参考・利用する独立プロジェクトであり、Jevの内部構造、学習データ、RLCDを知っている前提では設計しない。[S01][S02]

1.3 開発環境の前提

項目	前提と扱い
CPU / RAM	最近のユーザー申告：Ryzen 7 5800X / RAM 96GB
マザーボード	ASUS TUF GAMING X570-PLUS。ソフトウェア側に型番依存を持たせない
GPU	RTX A4000を使用。計画は16GB級GPUを1基使用する構成
GPU枚数	複数基の過去情報はあるが、現状は実機で確認。2枚必須にしない
OS	Windows系環境を想定。現在のOS版・WSL有無・Python環境はM0で確認
開発ツール	ローカルのCodexまたはAntigravity。特定バージョンの自動操作機能に依存しない
実行場所	ユーザーのPC。推論時のクラウドAPI呼び出しは行わない

既に動くCUDA/PyTorch環境があれば調査して活用する。ただし既存プロジェクトの環境を上書きしない。プロジェクト専用仮想環境を作り、必要なら既存環境と同じ動作実績のある組合せを再現する。最新バージョンへの一括更新はしない。

GPUが複数あっても通常の1プロセスでVRAMが自動的に合算される設計にはしない。GPU番号、空きVRAM、使用中プロセスを確認し、既存の画像生成やLLMサービスを勝手に終了しない。

2. 目的・非目的・完成の定義

2.1 初期版の目的

一つの質問につき2～16個の変候補を扱い、日本語入力から全候補の相対確率を返す。保存したモデルを再読み込みでき、同一評価セットで学習前後を比較できるようにする。モデルを信頼できる範囲と、まだ分からない範囲を可視化する。

使い道の例は問い合わせの振り分け、文章と条件の整合性確認、検索候補の選択、ゲーム内状態からの大まかな行動選択。最初は問い合わせ・明示ルール・本文と主張の関係など、正解を確認しやすい範囲で始める。

2.2 初期版で作らないもの

GUI、Web管理画面、ユーザー管理、認証基盤、課金、クラウド配置、データベース、ベクトルDB、RAG、マイクロサービス、Redis、Celery、Ray Serve、Kubernetes、分散学習、自動ハイパーパラメータ探索、独自プラグイン機構、オンライン自己学習、説明文生成、独自CUDAカーネルは作らない。

LoRA、量子化、ONNX、TensorRT、複数質問の本文共有計算、階層分類、連続スコア、複数ラベル、OpenAI互換API、C# SDKも初期必須にはしない。必要性が測れたものだけで追加する。

2.3 成果を三段階に区別する

段階	達成したこと	まだ言えないこと
推論試作	既存モデルをローカルで呼び、全候補のスコアを取得	独自学習済みモデルを作った、とは言えない
独自学習版	自分のデータで更新し、保存・再読込・未使用データでの比較を実施	あらゆる分野でJev相当、とは言えない
実用候補	対象範囲で品質・校正・速度・保留挙動を測定	個々の高確率回答が必ず正しい、とは言えない

3. 技術背景と判断を誤りやすい点

3.1 文章生成と候補採点の違い

候補採点モデルは、候補数K個のlogits（正規化前の実数スコア）を出す。Softmaxで合計1の分布へ変換する。JSON生成、語彙全体の逐次出力、回答文のパーズは不要になる。

ただし「出力が一度で出る」とこと「入力処理が軽い」とことは別。候補が増えれば入力長やattentionの計算量が変わる。数学・プログラム検証のような難しい判断が、二択にただでさえ容易になるわけでもない。

GLiClassは文章と候補ラベルを条件とする分類モデルを公開し、uni-encoder等の構成を提供している。任意の候補の意味を読んで採点する方向性が、ERABIの出発点になる。[\[S02\]](#)[\[S03\]](#)

3.2 正解ラベルだけでも確率出力を学習できる

正解候補のインデックスを用いるクロスエントロピーから開始できる。教師が出した分布を用いる場合は知識蒸留として扱う。教師分布の再現と、現実の正解率の再現は同じではない。[\[S07\]](#)

3.3 確率の意味を固定する

単一選択の $p[i]$ は、**その入力・質問・提示候補集合を条件として、候補*i*を選ぶモデルの分布**である。候補を追加・削除・言い換えすると変わり得る。未提示候補に対する絶対的な支持率ではない。

複数ラベルのSigmoid出力を合計1に割り直して、単一選択の確率と呼ばない。類似度、リランキングスコア、回答の事実性、主観的な好みも無断で同じ意味にしない。

3.4 校正と自信を分ける

Softmaxの最大値や分布の集中度は、そのまま回答の正解保証にはならない。校正では、例えば予測0.8の群が実際にどれほど正解しているかを確認する。温度スケールリングは初期候補だが、分布が変われば校正が崩れる可能性もある。[\[S08\]](#)[\[S09\]](#)

正の温度Tによる $\text{softmax}(z/T)$ は、同じlogits内の順位を変えない。したがって、温度調整だけでargmaxの分類正答率が改善したという報告が出たら、評価条件・マスク・丸め処理の変化を調べる。

3.5 仕様適合と正解は別

選択肢IDが壊れず返ることはソフトウェア側で保証する。しかし誤った候補を選ぶことはあり得る。形式の正常系テストに通ったことを、モデルの意味理解テストに合格したものと混同しない。

4. 初期の設計判断

項目	初期の決定	再検討する条件
タスク	単一選択のみ	独立した複数条件の要望が実利用から出た
ベース	GLiClassを先に測定	日本語・指示追従・依存関係に大きな問題がある
代替	日本語ModernBERT 130M + 共通採点ヘッド	第一候補の不足を具体的に確認した
学習	hard-labelの教師あり学習	soft-labelで改善する仮説とデータがある
推論	GPU1基、モデル1個、常駐1プロセス	実測上必要なときだけ増やす
データ保存	UTF-8 JSONLと小さなJSON/TOML	ファイル運用が現実には破綻した
HTTP	FastAPI + Uvicorn、localhostのみ	推論が完成し外部アプリから呼ぶ段階になった
テスト	純粋ロジック中心の小数テスト + GPUスモーク	実際のバグが再発防止を必要とする
実験管理	runs/<run_id>/ に記録	複人数・多数実験で管理が困難になった

独立した「将来のための抽象化」は作らない。最初はGLiClass用の薄いクラスまたは関数で十分。第二バックエンドを本当に実装するときに、必要な共通インターフェースだけを抽出する。

5. 全体の処理フロー

開発時

評価問題を用意 → 既存モデルを測る → 少量データで学習確認
→ 追加学習 → 開発用で選択 → 校正 → 最終評価 → 保存

実行時

context / question / choices
→ 入力検証と候補ID保持
→ ベースモデルに適合した入力整形・トークン数確認
→ 候補logitsを取得
→ 候補マスク・温度・Softmax
→ best候補と保留方針を計算
→ プログラムでJSONを組み立てる

学習と推論は同じ入力整形関数を使う。APIハンドラーの中に学習コードを入れない。予測結果を自動的に正解ラベルに変換して再学習しない。

6. 入出力仕様

6.1 リクエスト

```
{
  "schema_version": "1",
  "context": "昨日からログインできません。パスワード再設定も試しました。",
  "question": "最初に対応すべき担当窓口を一つ選んでください。",
  "choices": [
    {"id": "technical", "text": "技術サポート"},
    {"id": "billing", "text": "請求担当"},
    {"id": "sales", "text": "営業担当"},
    {"id": "insufficient", "text": "情報不足のため判断できない"}
  ]
}
```

6.2 レスポンス

以下は形式の例であり、モデルの実測値ではない。

```
{
  "schema_version": "1",
  "model_id": "erabi-example-untrained",
  "choices": [
    {"id": "technical", "probability": 0.87},
    {"id": "billing", "probability": 0.05},
    {"id": "sales", "probability": 0.02},
    {"id": "insufficient", "probability": 0.06}
  ],
  "best_candidate_id": "technical",
  "decision": {"status": "review", "reason": "policy_not_configured"},
  "calibration": {"status": "none", "artifact_id": null},
  "usage": {"input_tokens": 73, "truncated": false}
}
```

input_tokens は整形後の実際のモデル入力長であり、例の73は説明用。全候補をリクエストと同じ順序で返す。

best_candidate_id は採点上の最上位であり、自動実行の許可を意味しない。decision.status は accept または review。初期の既定は review。

calibration.status = applied は保存済みの温度を適用した事実だけを表す。未知の質問でも校正が保証されたという意味ではない。none のとき温度は1。どちらも確率分布は返せる。

6.3 検証条件

候補数は2～16。候補IDは1～64文字のASCII英数字・`_`・`-`で、リクエスト内で一意。候補textとquestionは空白だけにしない。contextは空文字を許す。NFC正規化と前後空白の除去で一致する候補textは拒否し、意味の近い候補はデータ品質検査で別に扱う。表記の意味を変え得る自動書換えはしない。

初期トークン上限は、**質問・候補・context・区切りを合計して512**。後にモデルと実機が対応すれば1,024へ上げる。リクエスト全体のUTF-8バイト上限は65,536。バイト制限とトークン制限は別に検査する。

初期版は `truncation=false`。上限超過は理由付きで拒否する。長文を無言で切る、候補だけを落とす、自動要約して別問題にする、`chunk`ごとの確率を合成することは禁止する。将来切り詰めを入れる場合も別仕様として明示する。

ベースモデルの候補マーカーと同じ特殊トークン文字列が入力に含まれる場合は、初期は `reserved_token_in_input` として拒否する。内部マーカーを一般文書に埋め込める状態を放置しない。必要な場合のみ、モデル・tokenizerを調べてエスケープ方式を追加する。

IDは出力への対応付け専用で、意味を判断するための入力に含めない。外部文字列を階層ラベル構文として解釈しない。候補IDが `technical` だから正解を推測できる、といったリークを避ける。

6.4 失敗時の扱い

状態	HTTP	内容
JSON・型・候補ID・候補数が不正	422	<code>invalid_request</code> 、問題のあるフィールド
バイト数超過	413	<code>request_too_large</code>
モデルの入力長超過	422	<code>input_too_long</code> 、上限と実トークン数
モデル未準備・GPU不足・処理中	503	<code>model_unavailable</code> / <code>out_of_memory</code> / <code>busy</code>
非有限logitsなどの内部異常	500	<code>invalid_model_output</code> 。本文やスタック詳細は返さない

失敗時に一様分布やランダム値を成功結果として返さない。CPUへ黙って切り替えてGPUと同じ性能を装わない。

7. 第一ルート：GLiClassを使う

7.1 最初に検証するチェックポイント

第一候補は `knowledgator/gliclass-multilang-mini`。必要な場合だけ、第二候補として `knowledgator/gliclass-instruct-base-v1.0` を比較する。二つを同時にGPUへ常駐させない。

`multilang-mini`のモデルカードはApache-2.0、約0.3B規模と表示されている。記載された20の学習言語に日本語は含まれない。これだけで日本語が使えないとは断言しないが、日本語対応を保証もしない。[\[S04\]](#) `instruct-base`も別の公開チェックポイントとして確認した。[\[S05\]](#)

実装時にはモデルIDだけでなく取得`revision`、`config`、`tokenizer`、ライセンス、依存するGLiClassの`commit`またはリリース版を記録する。`main` や `latest` を恒久的な再現性情報にしない。

7.2 最上位だけでなく全候補のlogitsを取得する

公開 `pipeline.py` の単一選択処理はSoftmaxを計算し、通常の予測リストには最上位のみを返す経路を持つ。全候補スコアの内部情報も存在するが、温度調整に必要な生logitsの公開取得方法は、採用`revision`で確認する。[\[S11\]](#)

ERABIでは、整形・tokenizeの既存機能を活用し、モデルforwardの `logits` を取り出す薄いadapterを優先する。既存APIで十分ならそれを使い、非公開APIに依存する場合はその箇所を一ファイルに隔離して版を固定する。

通常パイプラインの出力を見て、欠けた候補に0を補う実装は禁止。Sigmoid値を確率の代わりに合計1へ割り直すこともしない。`logits`へSoftmaxを1回だけ適用する。

7.3 学習ラベル経路を確認する

確認した `data_processing.py` は、単一選択で `true_labels[0]` から正解インデックスを作り、辞書形式の数値は複数ラベル用経路で処理する。したがって単一選択のsoft targetは、JSONへ数値を入れるだけで自動的に実現するとは限らない。
[S12]

初期はhard targetだけにして、必要なときに全候補分布用のcollatorとlossを追加する。既存のlossが可変候補数やpaddingを期待通り処理しているかも採用revisionで確認する。[S13]

7.4 既存の学習例をそのまま本番データへ使わない

確認した `train.py` はデータをシャッフルして90%・10%に分ける。ERABIでは事前に定めたsplitを読み、シナリオ由来の重複が混ざらないようにする。[S14]

GLiClassの追加機能をすべて取り込まない。階層ラベル、RAG、Ray、streaming、任意の高速attention拡張は初期には無効。ローカル用薄いwrapperを作るためだけに上流全体をforkしない。

8. 第二ルート：日本語エンコーダーから判断モデルを作る

8.1 切替条件とベース

日本語の簡単な問題で大幅に失敗する、指示を変えても回答が変わらない、依存関係の修正が過大になる、といった測定結果が出た場合に検討する。最初から二方式を完成させる計画にはしない。

候補は `sbintuitions/modernbert-ja-130m`。モデルカードでは日本語・英語で事前学習された132Mパラメータのモデル、最大8,192トークン、MIT Licenseとされる。これはマスク言語モデルの基盤であって、そのまま任意判断ができる完成済み分類器ではない。[S06]

GLiClassの重みの一部を交換するのではない。別のエンコーダーを基盤に、候補表現と採点を学習する独立実装になる。TransformersのModernBERTサポートとattention実装を実機で確認する。[S19]

8.2 最小の構造案

入力は質問、context、各候補を一つの系列に並べ、同じ候補マーカーで区切る。独自の番号別出力ヘッドを作らず、候補ごとに同じMLPを適用する。

```
[QUESTION] 判断基準 [CONTEXT] 状況文
[CHOICE] 候補A [CHOICE] 候補B ...
↓ encoder
候補マーカー位置のhidden stateを取得
↓ 共有 Linear → GELU → Linear
各候補のlogit
```

候補マーカーが候補テキストの情報を参照できること、長いcontextとの情報伝達が足りることは実測する。ModernBERTの局所・大域attention等の内部を独自に改造するところから始めない。

追加マーカーはtokenizerに登録し、embeddingサイズを変更して保存する。学習前後・再読込後に、マーカーID、候補数、候補位置が一致することを確認する。IDの一部を固定語彙へ転用しない。

8.3 バッチのtensor契約

値	形状・意味
input_ids / attention_mask	[B, L]。整形後の系列
choice_positions	[B, Kmax]。各候補のマーカ位置
choice_mask	[B, Kmax]。存在する候補だけtrue
logits	[B, Kmax]。padding候補を後で除外
hard_target	[B]。各行の正解候補インデックス
soft_target	[B, Kmax]。将来用。paddingには質量を置かない

最初は候補マーカ位置の表現だけを使い、複雑なcross-attention scorerや二重encoderは追加しない。表現能力が不足した証拠が得られた場合だけ、候補span pooling等を比較する。

9. データ設計と分割

9.1 共通JSONL形式

```
{
  "schema_version": "1",
  "id": "route-0001",
  "group_id": "scenario-route-001",
  "task_family": "support_routing",
  "language": "ja",
  "context": "ログインできません。",
  "question": "最初に対応する担当を選んでください。",
  "choices": [
    {"id": "technical", "text": "技術サポート"},
    {"id": "billing", "text": "請求担当"}
  ],
  "target": {"kind": "hard", "choice_id": "technical"},
  "source": {"kind": "authored", "reference": "internal-001"},
  "quality": {"label_status": "unreviewed"}
}
```

この文書と添付examplesは作成例であり、人手で独立検証済みのgoldデータではない。unreviewed を初期値とし、レビュー後に human_verified または実際にプログラム検証した program_verified へ変える。LLMが作成しただけで human_verified を付けない。

追加する場合のsoft targetは、{"kind": "soft", "probabilities": {"a": 0.7, "b": 0.3}}。候補ID集合と完全一致し、全値が有限・非負・合計1（許容誤差1e-4）でなければ拒否する。欠損候補を無断で0補完しない。丸め誤差の範囲だけ正規化し、修正した事実を記録する。

信頼できる正解と教師分布を併用する場合は、独立した verified_choice_id と teacher の由来を持たせる。教師と正解が食い違った行は自動採用せず確認対象にする。

9.2 同じ問いの派生を同じgroupへまとめる

同一元文書、シナリオ、言い換え、翻訳、候補順序違い、対照的な質問違いは同じ group_id に入れる。元の生成seedやテンプレートIDもsource側へ記録できるようにする。

分割は先にgroup単位で行い、その後に学習用だけを拡張する。group IDだけでなく、入力正規化hashの重複も確認する。テンプレートを数値だけ変えた大量生成が評価の大部分を占めないよう、テンプレート族を外した評価も用意する。

9.3 四つの用途別split

split	使い道	禁止事項
train	モデル重みを更新	final_testを取り込まない
dev	モデル・学習率・終了時点の選択	成績を最終成績として宣伝しない
calibration	温度と保留閾値の調整	重みやモデル候補の探索に流用しない
final_test	最後の固定評価	見た誤りを修正したモデルを同じ意味で再評価しない

初期の20～100問は配線確認・開発用。品質比較のための固定devは200～300問から作る。最終的にはdev・calibration・final_testをそれぞれ独立groupで確保するが、件数だけを満たすために未検証例を大量投入しない。小規模時は「傾向確認」として報告し、実用誤り率を保証しない。

final_testで発見した問題を次版のtrainへ移す場合、そのテストセットは以後開発に使った扱いにし、新たな未使用評価を用意する。公開モデルの事前学習・事後学習への混入は完全には確認できないため、「ゼロ汚染」とは主張しない。

9.4 データの種類

最初は問い合わせの意図、与えたルールへの適合、本文が主張を支持・否定・判定不能のいずれか、を扱う。同じcontextと候補に対し「最安」と「最速」で正解が変わる対照例を含める。

否定・引用・数値境界・単位・情報不足・もっともらしい誤候補・本文中の偽指示を追加する。ただしプログラムで正確に計算できる仕事を本番でわざわざモデルへ置き換えることは推奨しない。ルール問題は学習と検証の材料として使い、実システムでは確定計算をコードへ残す。

日本語の外部データ候補にJGLUEがあり、JNLIやJCommonsenseQA等を調べられる。データごとの用途・ライセンス・既存の分割に従う。選択肢IDの文字を分類クラスとした全体macro-F1は、問題ごとに意味が異なるため適切ではない。
[S20]

10. 教師データの作成と品質管理

10.1 優先する供給源

第一に、自作の明示ルール・状態からコードで正解を決めた例。第二に、利用可能な既存データ。第三に、許諾を確認した教師LLMによる生成・注釈を使う。教師はデータ作成時の補助であり、推論時の必須依存にしない。

コードで作った構造化シナリオをLLMに自然文へ言い換えさせる場合、正解は元の構造化データから決める。ただし言い換えで否定や条件が落ちる可能性があるため、原データと文章の整合性を検査する。機械的チェックを通ったことと、自然文の完全な品質保証は別である。

10.2 外部APIに関する既定値

外部API予算は0、送信は無効。設計書を渡しただけで、課金データ生成が許可されたと解釈しない。利用者が提供元・モデル・上限額・送信可能データを指定した後でのみ有効化する。

以前の会話で検討した提供元が、現在も蒸留・競合モデル学習を許可していると決めつけない。実装時に対象エンドポイントの規約を確認する。Jev APIから大量に模倣データを取得することや、利用規約を回避することは計画に含めない。

認証情報は環境変数等で扱い、データ・ログ・Gitへ保存しない。個人情報、業務秘密、APIキーを教材へ混ぜない。エージェントによる開発自体の送信範囲と、ERABI推論のオフライン性は別なので、秘密データを開発エージェントに読ませない配置も検討する。

10.3 保存する由来と教師出力

source には出所、取得日、元データID、ライセンス確認先、テンプレートID、生成seedを必要な範囲で記録する。教師利用時はprovider、正確なmodel ID、prompt版、生成設定、実際の入出力トークン数を追加する。

教師の根拠文は品質確認用の別情報として保存してよいが、生徒の入力へ混入させない。正解ラベル名をcontext末尾に付けるようなリークも防ぐ。教師の自己申告確率は teacher_reported、logits由来ならその旨を明記し、同じ品質とみなさない。

料金の見積もりは実際の入力・出力単価とトークン数から計算し、教師の単価は設定値にする。再試行も予算へ含め、回数上限を設ける。大きな抽象化は不要で、採用した提供元一つの短いスクリプトで始める。

10.4 人手確認を使う場所

評価・校正用は信頼できる正解を優先する。学習用でも、無作為サンプル、教師同士の不一致、高確率で誤った例、境界事例を確認する。教師同士の一致だけを正解保証にはしない。

異常行は rejected.jsonl と理由を保存し、黙って消さない。データ全体の言語、タスク、候補数、正解位置、入力長の分布をJSONへ出す。可視化ダッシュボードは作らず、表と少数の必要な集計で十分。

11. 学習設計

11.1 開始条件とデータ量

最初に数十～100件の正解が明確な例へ意図的に過学習させる。この段階の目的は学習能力の評価ではなく、正解対応・勾配・保存再読込の配線確認である。

続いて1,000件、必要なら1万件へ増やす。その後5万～10万件以上を検討する。これは必要量の断定ではない。1万件で改善しない場合は、件数を増やす前にラベル品質、入力整形、候補マスク、タスクの難度を確認する。

GLiClassを追加学習する場合は既存の分類学習を利用する。日本語ModernBERTから新設する場合は、候補採点自体を学ぶ負担がある。同じ追加データ量で同じ性能になるとは仮定しない。

11.2 損失

正解候補yに対する基本損失は $L_{\text{hard}} = -\log p[y]$ 。有効候補だけからなる予測pに対して計算する。まず既存モデルの分類設定と合わせ、数値が有限であることを確認する。

soft target qを使う場合は $L_{\text{soft}} = -\sum(q[i] * \log p[i])$ 。教師分布の情報しかない例と、人が確認したhard labelを持つ例を区別する。両方あるときだけ、例えば $L = (1-a) L_{\text{hard}} + a L_{\text{soft}}$ を比較する。a=0の比較を残し、初期はsoft-label学習を実装しない。[S07]

既に確率になっているqへ再びSoftmaxをかけない。温度付きlogits蒸留の式を、教師logitsのないデータへそのまま流用しない。KLとsoft cross-entropyは教師固定なら生徒に関する勾配が同じ方向になるが、損失値自体は同じとは限らない。

11.3 可変候補数のマスク

候補数が違う例を同じバッチへ入れる場合、padding候補をSoftmax分母と損失から除外する。無効な位置はfloat32上で十分小さい値または適切なmaskで扱い、出力確率を0にする。

soft lossで $0 * -\inf$ によりNaNを作らない。有効候補位置でのみ積和を取る、または安全な有限log値を使う。全候補mask=falseの行は入力段階で拒否する。

候補順序のaugmentationではtext・ID・hard target・soft targetを同じpermutationで並べ替える。順番だけを変えた例は独立標本数に数えず、同じgroupを維持する。

11.4 初期学習設定案

項目	出発点	注意
長さ	全入力512 tokens	候補と質問も含む
候補数	最初2～8、後に16	API上限と評価対象範囲を合わせる
micro batch	2	OOM時は1へ。実測後に増やす
gradient accumulation	8	batch=2なら実効16例相当を基本とする
encoder学習率	2e-5	devで必要最小限比較
新設ヘッド学習率	1e-4	既存ヘッドへ機械的に適用しない
optimizer	AdamW	標準実装を使う
epochs	最初1、必要なら2～3	無条件に全パターンを総当たりしない
precision	安定する混合精度	BF16対応と実際の演算を確認。FP16はscalerを使う
gradient clip	norm 1.0を出発点	clip対象とscalerの順を標準手順に合わせる
seed	固定値42を初回例とする	最終候補のみ追加seed比較を検討
dataloader workers	0から開始	Windows・WSL差異とRAM使用量を抑える

初回はdry-runの最大step数を明示してから動かす。長時間・大量データの学習を自動ループで始めない。optimizerの状態は最初のstepで確保される場合があるため、VRAM確認はforwardだけでなくbackwardとoptimizer.stepまで含める。

全層学習が小型モデルで成立するなら、LoRAを入れる理由はない。VRAMが足りない場合は、長さとbatchを減らし、次にgradient checkpointing等を比較する。量子化学学習や分散処理へすぐ進まない。

11.5 再現性・保存

Python、NumPyを使用する場合はNumPy、PyTorchのseedを固定し、split、依存版、CUDA、GPU、precision、attention方式、model revisionを記録する。異なる環境でbit単位の一貫性を保証しない。

推論用保存物はweights、config、tokenizer、入力整形version、元モデル情報、学習データmanifest、評価要約。学習再開用optimizer状態は別扱いとし、必要なときだけ保持する。外部から受け取ったpickle等を無条件で読み込まない。

各runの別ディレクトリへ保存し、既存の良好なモデルを上書きしない。最初はモデルの自動昇格やregistryを作らず、採用したディレクトリを設定で指定する。

12. 校正・保留・自動処理

12.1 温度スケールリング

モデル選択後に重みを固定し、calibration splitのlogitsを保存する。正のスカラーTを、calibration上のNLLが小さくなるよう最適化する。例えば $T = \exp(t)$ と置き、非有限値や極端な値を検査する。まず一つのTだけを扱う。[S08]

出力は $\text{softmax}(z/T)$ 。T=1との差をNLL、Brier、信頼度帯ごとの正解率で確認する。校正用データで改善してもfinal_testで改善するとは限らない。Tを学習したこと自体を成功条件にしない。

12.2 校正ファイルの結び付け

calibration.json にはtemperature、モデルweights識別子またはhash、tokenizer/formatter版、precision、入力上限、calibrationデータhash、適用範囲の説明、作成日時を保存する。

異なるweights・入力整形版の校正を誤適用した場合は起動を失敗させる。校正なしで起動するには明示設定を使い、status=none を返す。量子化・モデル更新・precision変更後は再評価し、校正を引き続けると仮定しない。

未知タスクを一般的に検知する仕組みは初期版にはない。applied は適用事実であり、学習分布内であることの判定ではない。入力を見ただけで万能な未知検出ができるようなフィールドは設けない。

12.3 保留の方針

既定では decision.status=review。実利用範囲を絞った検証後、サーバー側の設定で $p_{\max} \geq \text{threshold}$ の採用方針を有効にする。marginやentropyによる複数閾値は必要性を見てから追加する。

同率最上位の場合、表示上のbestは入力順で決めてよいが、自動処理はreviewにする。単に確率0.95を超えたから安全とは判断しない。呼び出し元が任意の「安全モード」を指定して閾値を下げるAPIは作らない。

insufficient などの候補を自動追加しない。候補集合が変わると問題の意味と分布が変わる。利用者が明示した「該当なし」「情報不足」と、モデルの低確信による保留は分ける。必要に応じてサーバー側の review_choice_ids に情報不足IDを設定できるが、初期はデフォルト全reviewで足りる。

12.4 評価する運用指標

coverageは自動採用件数/全件数、selective riskは採用件数中の誤り率。採用0件ならriskは0%ではなく未定義としてnullと件数を返す。閾値ごとに両方を示す。

小さなサンプルで誤り0件でも、誤り率0%が証明されたわけではない。独立なn件で誤り0件だった場合の片側95%上限は、二項モデルの仮定下で $1 - 0.05^{1/n}$ 。例えばn=100では約2.95%。同一シナリオの言い換えを独立な100件とみなすのも不適切である。

厳密な運用品質を検討する段階で、独立group単位の集計や区間推定を追加する。初期から統計フレームワーク一式は実装しない。

13. 評価設計

13.1 ソフトウェア検証と意味の検証を分ける

配線が正しいかはユニット・統合テストで確認する。モデルが文を理解したかは固定データセット上で測る。学習モデルへ「この日本語なら必ず0.95」といった数値を要求するユニットテストは書かない。

13.2 必須の品質指標

指標	目的・注意
Accuracy	1問1正解での正答率。タスク別・候補数別も見る
NLL	正解に確率を割り当てられたか。計算時のclip規則を固定
Brier	$\text{mean}(\sum_i (p_i - y_i)^2)$ と定義を固定。候補数別も報告
Reliability bins	pmaxの範囲ごとの件数・平均pmax・正解率
Coverage / risk	保留と自動処理の品質を同時に評価
誤り例	高確率誤答、情報不足、指示違い、長文を区別

ECEは必要になれば上記binsから算出できるが、bin分割や標本数への依存があるため唯一の品質指標にしない。macro-F1を使う場合は固定の意味クラスを持つ特定タスク内に限定する。

13.3 比較対象

最初は候補一様分布、学習前の同一チェックポイント、追加学習後のモデルを比較する。必要なら既存LLMの候補採点を一つ加えるが、比較のためだけに大きなLLM基盤を構築しない。

ランダム選択の期待正答率は問ごとの $1/K$ の平均で、候補数混在時に単一の25%等を固定しない。単純ルールで解けるタスクにはルール方式も残す。

同じ問題集合、同じ長さ制限、同じ候補、同じ評価定義を使う。モデル都合で除外した問題は除外率を報告し、長い問題を黙って捨てた成績を全件性能と比較しない。

13.4 変形・耐性の検証

候補順序を変え、IDで元へ戻した後のbest一致率と分布差を測る。これはモデルの意味的な安定性の評価であり、完全一致を必須にするユニットテストではない。

候補IDだけを変更した場合は、IDをモデルへ渡さない設計なので分布が同じになることを実装テストで確認できる。本文の偽指示、引用内の命令、長い無関係文、否定、数値境界、表記揺れについても小さな固定セットを用意する。

未知タスクはtask_familyを丸ごとtrainから外した評価で測る。「既知タスクの新しい文章」と「初めての仕事」を同じ一般化能力として集計しない。

13.5 採否の考え方

最初の改善目安は、devで学習前より正答率が数ポイント改善する、または正答率を概ね維持しながらNLLを改善すること。小標本の差は偶然もあり、機械的な合否保証にしない。

校正後NLLが悪化した、重要タスクが大きく退行した、情報不足を高確率で断言する、といった場合は採用を保留する。全面改善がなければ、特定用途へ範囲を狭める選択も有効である。

14. 過剰にしないテスト方針

14.1 原則

網羅率の数値目標は設けない。単純なgetter、設定読み込みの各行、PyTorch内部、外部SDKの全メソッドをテストしない。モックで内部実装を固定せず、入力と結果の契約をテストする。

初期のユニットテストは**6～8個の責務**を、少数のパラメータケースで確認する程度を目安とする。件数は上限規則ではないが、テストを増やす前に「どの実害を防ぐか」を説明できるようにする。通常のテスト実行でネット接続、大きなモデルダウンロード、学習、GPU確保を行わない。

14.2 優先するテスト

責務	小さく確認する内容
入力検証	重複ID、空候補、上限超過、特殊マーカー混入を拒否
IDと候補の対応	並べ替え後も正解・確率を正しいIDへ戻す
Softmaxとmask	2候補/3候補混在でpaddingに確率が付かない
損失	hard labelと対応するone-hot soft targetの整合、NaN防止
split	同じgroupと同じ元データhashが複数splitへ入らない
校正結合	T>0、T=1の整合、モデル不一致の校正を拒否
保存再読込	小さな自作ヘッド等の状態と設定を復元できる
API契約	fake scorerで全候補返却、review既定、異常時に成功扱いしない

soft loss未実装の段階でそのテストだけ先に大量に作らない。校正やAPIも該当フェーズで追加する。GPUなしの小さなtensorと、手で計算できるlogitsで十分な箇所はそれを使う。

14.3 GPU検証は別コマンド

gpu_smoke は明示実行する検証として扱い、実モデル1問のforward、全候補logits、保存再読込の整合を確認する。必要になった学習段階で、数十件の過学習確認を別実行する。

pytestを使うなら gpu / slow を明示し、通常実行から除外する。実モデルを使わない通常テストは、開発時に繰り返し回せる規模に保つ。CPU/GPUの推論値は適切な許容誤差を用い、bit一致を要求しない。

14.4 回帰テストを増やす基準

実害のあるバグが発生した、入力仕様が追加された、保存形式が変わった場合に、その差分を一つの小さなテストへする。学習結果が変わっただけで大量のgolden fileを更新する運用は避ける。

15. ローカルAPIと安全な運用

15.1 最小の構成

Pythonから直接推論できてからAPIを追加する。POST /v1/choice と GET /health の二つで開始する。FastAPI + Uvicorn、1 worker、127.0.0.1:8765を既定とする。

起動時にモデルを一度だけ読み、リクエストごとにロードしない。複数workerはモデルのメモリを複製し得るため、CPUコア数に合わせて自動増加させない。[S17]

APIで任意のモデルURL・ローカルファイルパス・Pythonコード・プロンプトからの実行コマンドを受け取らない。モデル選択はサーバー起動設定のみ。モデル出力は推奨であり、OS操作・送金・ゲーム内操作等を直接実行する権限を持たせない。

15.2 同時実行と失敗

初期は同時GPU推論1件。別リクエストは無制限にキューへ積まず、処理中を示す503で返してよい。推論部分をAPIイベントループ上で直接長時間塞がないよう、単一実行ガードと標準のthread実行等で小さく構成する。

リクエストタイムアウトやクライアント切断が、GPUカーネルを強制停止したことを意味するとは限らない。ワーカーを大量に作り直して回避せず、実行が終了するまでガードを保持する。OOMを無限再試行しない。

15.3 ログとプライバシー

既定ログはrequest ID、model ID、トークン数、候補数、経過時間、成功・失敗理由だけ。context・question・ラベル本文を通常ログへ保存しない。errorのスタックに秘密入力が出ないようにする。

開発用の予測保存は、匿名化済み評価データに対して明示コマンドで行う。ブラウザUIを作らないためCORSは既定無効。localhostでも他のローカルプロセスから呼ばれ得る点は認識し、外部公開は別課題として扱う。

15.4 完全オフラインの確認

モデル、config、tokenizer、必要な依存を事前取得し、推論時はlocal pathで読む。Hugging FaceはHF_HUB_OFFLINE=1 と local_files_only=True等を案内している。設定だけで完了とせず、ネット接続を遮断した環境で起動・推論を一度確認する。[S10]

APIのSwagger UI等は外部CDNへ依存する場合があるため、厳密なオフライン運用では/docs / /redoc を無効にするか静的資産をローカル化する。初期は無効でよい。curlやC# HttpClientからのJSON呼び出しで確認する。

16. コード構成と依存関係

16.1 M1時点の最小構成案

```
erabi-local/
  AGENTS.md
  KICKOFF.md
  README.md
  STATUS.md
  docs/ERABI_DESIGN.md
  pyproject.toml          # 実装開始時に作る
  src/erabi/
    __init__.py
    __main__.py           # argparseでよい
    schema.py             # 入出力契約
    inference.py          # GLiClassの薄いadapterと後処理
    evaluate.py           # JSONL入出力と指標
  tests/
    test_contracts.py
    test_scores.py
  examples/
  models/                 # Git対象外
  runs/                   # Git対象外
```

M2で data.py、train.py、M3で calibrate.py、serve.py を必要に応じて追加する。代替モデルが必要なら model.py を追加する。全ファイルを空クラスで先に埋めない。20行程度の処理を複数レイヤーへ分割しない。

16.2 ライブラリ選択

PyTorch、Transformers、GLiClassを基盤とし、tokenizerが要求する依存を追加する。API・型検証段階ではFastAPI / Uvicorn / Pydanticを使える。JSONL・CLI・設定は標準ライブラリのjson / argparse / tomlib等を優先し、必要なら既存環境のパッケージ管理方式に合わせる。

実験データがファイルで扱えるならpandas、datasets、scikit-learnを必須にしない。ただし既存コードが有効に使っているものを、依存削減だけのために大規模再実装もしない。

パッケージ管理は既存方針があればそれに従う。新規ならvenvとpipを出発点にしてよい。動作確認した依存を固定し、GPU版PyTorchの取得元も記録する。uv、Poetry、Dockerを同時導入しない。

16.3 実験の保存先

```
runs/<run_id>/
  config.json
  environment.json
  dataset_manifest.json
  metrics.json
  predictions.jsonl
  notes.md
  checkpoint/             # 学習したrunのみ
```

命名と構造を一度決めれば、追加の実験追跡サーバーは不要。ログを無制限に残さず、モデルを削除する場合は利用者の作業成果を確認する。古いrunを勝手に自動削除する仕組みは作らない。

17. 環境確認とセットアップの手順

17.1 M0で読むもの

既存リポジトリの指示、作業ツリー、Python環境、依存ファイル、GPU使用状況を確認する。既存の変更を上書きしない。Gitがない場所に勝手にリモートを作らず、公開pushをしない。

読み取り中心の確認例は次のとおり。存在しないコマンドは環境に合わせる。

```
python --version
python -m pip --version
nvidia-smi
python -c "import torch; print(torch.__version__); print(torch.version.cuda); print(torch.cuda.is_available())"
```

torchが未導入で失敗することは、GPUが存在しない証拠ではない。OS、実行シェル、仮想環境のpath、RAM、空きディスク、GPU名・枚数・空きVRAMを environment.json に残す。

17.2 WindowsとWSL2の選択

現在のWindowsネイティブ環境で候補モデルが動くなら、それを優先してよい。依存やattention実装で問題が出る場合はWSL2/Linuxを選択肢にするが、最初から環境の全面移行を強制しない。

WSL2を使う場合、CUDA用のWindowsドライバーを使い、WSL内にLinuxディスプレイドライバーを重ねて入れない。NVIDIAの公式手順でも注意されている。[\[S15\]](#) 管理者権限、ドライバー変更、OS設定変更を必要とする操作は利用者に確認する。

エディターとPythonの実行場所がWindows/WSLで異なる場合、ワークスペースpathとlocalhost接続を小さな確認で揃える。接続できないからといって0.0.0.0公開やファイアウォール開放へ直行しない。

17.3 インストールとモデル取得

実装時にモデルID・revision・必要な依存・取得容量・保存先を確認する。公開モデル取得と課金API実行は別権限として扱う。ディスク不足のときに他プロジェクトのcacheを削除しない。

trust_remote_code=True を無条件に使わない。必要なら取得したコードと由来を確認して記録する。モデルやデータのライセンス未確認のまま再配布しない。プロジェクト本体のコードライセンスも、第三者モデル・データとは分けて決める。

18. 速度・VRAM・高速化の検証

18.1 測定条件

初回ロード時間、warm状態の推論時間、tokenize込みの時間、HTTP込みの時間を分ける。GPUの計算は非同期なので、モデル区間の計測にはCUDA eventまたは適切な同期を使う。APIの実測は呼び出し側の経過時間で測る。

同一runにGPU名、使用中の他プロセス、dtype、モデルrevision、batch、長さ、候補数、warmup回数と測定回数を残す。p50、p95、処理件数/秒、最大VRAMを報告する。最初はwarmup5回、測定30回などの小さな測定から始め、最終値は十分な回数で再確認する。

18.2 小さく始める測定行列

全組合せのベンチマークを最初から走らせない。初回は全入力512 tokens以内・候補4個・batch1を測り、必要なら短い入力/候補16個/1,024 tokensへ広げる。

候補数を増やす比較では、候補文の長さ合計token数も記録する。「候補8個」という情報だけでは計算量を比較できない。同じcontextを複数質問で別々に読む場合、通常のbatch処理では本文の計算が共通化されるわけではない。

18.3 目標値と採否

初期の応答時間は測ってから目標を決める。A4000で何ms、Jevより何倍速い、といった未確認の数字を設計に埋め込まない。仮の使い勝手目標としてp95が1秒以内等を置くことはできるが、達成保証ではなく比較のための目安とする。

VRAM不足では、まず他プロセスへの影響を確認し、入力長・batchを減らす。CPU fallbackは明示設定でのみ可能とし、その結果をGPU測定と混ぜない。GPUがない環境でも契約・データ処理・数値テストは進める。

18.4 改善の優先順位

一回だけのロード、evalモード、inference_mode、不要な逐次処理の削減、padding量の削減、安定した混合精度を先に行う。その後、ボトルネックが確認された場合だけbatch化・量子化・コンパイル等を比較する。

新しい高速化方式を入れたら、同じ評価セットで順位・NLL・校正・長さ制限も確認する。速度だけ改善し意味の正しさが崩れたものは採用しない。最適化後のモデルについて校正の適用可否を再確認する。

19. 実装フェーズと完了条件

M0：現状確認と作業基盤

成果物は環境記録、既存変更の確認、最小の依存方針、最初に使うモデルの選定。既存プロジェクトに合った仮想環境を選び、公開情報と実機の相違を記録する。

完了条件は、モデル取得・推論をどこで行うか、どのGPUを使うか、何が未確認かが分かること。ドライバー変更や課金を行わなくても進められる部分を進める。

M1：既存モデルを呼び、比較できる状態にする

入力検証、GLiClass adapter、全候補logits、正規化、JSONL推論、基本指標、少数のCPUテストを作る。まず一問で動かし、次に20～100件の開発用例で結果を保存する。

完了条件は実モデルから有限の全候補分布を得て、ID対応が正しく、上限超過を検出でき、実測結果と環境が保存されること。ここではまだAPI、追加学習、大量合成、校正は不要。

日本語が不足する場合は、その具体例を示す。第二の既存チェックポイント比較がModernBERTへの切替を提案し、両方を無条件に完成させない。

M2：独自学習を成立させる

group単位split、学習用collator、hard-label損失、保存再読込、100件程度への過学習、1,000～1万件の追加学習を段階的に実施する。M2内でも、まず最小の学習確認を終えてから増量する。

完了条件は、学習したweightsを再読込でき、未使用devで学習前との比較が出ること。改善しない場合も原因を分類して記録し、「学習済みだから高性能」とは報告しない。

M3：校正とローカルAPI

モデルを選び、独立calibrationで温度を決め、別final_testで評価する。review既定のAPI、モデルに結び付いた校正ファイル、オフライン起動、基本的なエラー処理を追加する。

完了条件はモデルと校正の不一致を拒否し、ネットなしで推論し、coverage/riskと速度が確認できること。対象範囲に対する自動採用条件が未検証ならreview既定のままでよい。

M4：必要性が確認された拡張

soft-label蒸留、日本語ModernBERT別版、より広いタスク、複数質問、量子化、SDK等から、測定した不足に対応するものを一つ選ぶ。M4は最初の依頼範囲に自動的に含まない。

フェーズの共通ルール

計画説明だけで止まらず、権限内でできる小さな実装と実測まで進める。ただし実行できない環境でfake値を実測と呼ばない。大きな処理を無断で開始しない。終了時に STATUS.md へ完了・未完了・実行したコマンド・次の一手を短く記録する。

20. Codex / Antigravityへの引き渡し

20.1 共通方式

プロジェクトのルートに AGENTS.md と KICKOFF.md を置く。最初の依頼では、この二つを読むよう明示する。巨大な設計書を常時ルールに全文複製せず、必要な節を読み込む方式にする。

Codexはプロジェクトの AGENTS.md を指示として読む仕組みを公開している。親ディレクトリや上書き用指示の影響もあるため、実装前に適用中の指示を確認する。[S16]

Antigravityの公式資料では、workspace rulesに .agents/rules/ を使い、従来の .agent/rules/ との互換も記載されている。添付の .agents/rules/erabi.md は短い参照用ルールである。バージョン差があるため、Customizationsで適用状態を確認し、必要ならAlways onに設定する。[S18]

Antigravityが AGENTS.md を自動読込すると仮定しない。最初のプロンプトで明示する方式なら、個別設定に依存しにくい。全プロジェクトへ効くglobal設定は変更しない。

20.2 エージェントへ渡す初回依頼

このフォルダのAGENTS.mdとKICKOFF.mdを読み、ERABIを実装してください。
まずはM0とM1だけを進めてください。
既存環境と作業中の変更を確認し、推論・全候補確率・小さな評価を実測してください。
大規模学習、課金API、UI、分散処理はまだ作らないでください。
テストは候補ID対応、確率とマスク、入力検証など、実害の大きい箇所を絞ってください。
完了した作業、実行した検証、未実行の検証、次の一手をSTATUS.mdに残してください。

20.3 過剰設計を止める判断

「二つ目の利用先がない共通基盤」「未使用の抽象クラス」「将来のサービス群」「一度しか使わない処理のプラグイン化」「デモ用UIを先に作る」「100%カバレッジを追う」場合は止める。

変更ごとに、今のフェーズのどの完了条件へ寄与するかを確認する。既存ライブラリの一箇所を小さく包めば足りる問題を、新しいフレームワークへ置き換えない。人間向けに設計変更理由を数行残せば、初期には大量のADR文書も不要。

21. 主な失敗モードと対処

兆候	原因候補	最初の対処
全候補が出ない	既存pipelineがtop1だけ返す	forward/logits経路を確認
確率が不自然	二重Softmax、Sigmoidの再正規化	生logitsからの処理を点検
batchで結果が壊れる	padding候補の混入、候補ID対応ミス	2候補/3候補の手計算例で確認
長文だけ妙に自信が高い	truncationで条件が落ちた	長さ上限を明示し拒否
学習で改善しない	日本語・指示不足、ラベル誤り	小さな過学習と代表誤りを確認
評価だけ非常に良い	シナリオ・正解のリーク	group、hash、入力への根拠混入を点検
高確率で誤る	校正不良、未知タスク、候補集合変化	範囲を限定しreviewへ
校正後に急に悪化	別モデルのTを適用	artifactとweights識別子を照合
OOM・OSが重い	複数worker、他GPU用途、optimizer状態	1 worker、短い入力、小batchへ
ネットなしで起動しない	tokenizerやconfigが未取得	全依存をlocal pathから再確認
開発が肥大化する	API/UI/学習/評価を同時に作る	現フェーズの完了条件へ戻る

モデルがどれだけ高確率でも、重要な操作を無条件実行させない。初期の統合先では、選択結果を表示・記録する段階から始める。ゲームなら移動・衝突・経路探索は通常コードへ残し、ERABIには大まかな選択だけを渡す。

22. 将来拡張の設計メモ

以下は、今すぐ実装しないために整理するものである。

複数ラベルは各候補への独立Sigmoidと複数正解データを使い、単一選択の合計1契約とは別エンドポイントまたはtask種別にする。真偽判定は二択でも開始できるが、「真である」「本文からは分からない」を混同しない。

採点はまず定義した段階への分類を考える。期待値を使う場合、その段階を等間隔の数値として扱ってよいかを確認する。主観的な品質評価に、小数点以下の精密な真値があるように見せない。

複数質問は、最初は独立例のbatchでよい。本文共有の一回encodeへ進む場合、質問グループごとのmask、候補間の干渉、学習形式を新たに検証する。「一回呼出し」と「一問相当の計算量」を混同しない。

量子化・ONNX等は実測でボトルネックがあるときだけ検討する。自動学習は、正解確認・評価・人間の昇格判断を残したoffline更新から始め、推論ログを無条件に学習へ戻さない。

23. 完成前のチェックリスト

初期の推論版

- 実モデルで日本語一問を処理し、実測と例示を区別した。

- 全候補のIDと確率が戻り、候補数・入力長・特殊マーカーを検証した。
- 評価時に候補集合と対象件数が記録され、上限超過の除外を隠していない。
- 通常テストがGPU・ネット・大量モデルを要求しない。
- 既存環境や利用者の変更を壊していない。

学習版

- 学習と推論の入力整形が一致し、候補マスクと正解が合っている。
- train/dev/calibration/final_testの独立groupを確認した。
- 自分で更新したweightsを保存・再読み込んで評価した。
- 学習前・学習後の差を同一条件で示し、悪化も隠していない。
- データ・モデル・コード・依存の由来が追える。

実用候補版

- 校正が対象モデルと結び付いており、温度適用と正解保証を区別した。
- review既定で、採用率と採用中の誤り率を併記した。
- オフライン起動・推論を実際に確認した。
- p50/p95とVRAMを実機条件付きで記録した。
- 課金APIや外部公開を勝手に有効化していない。

24. 参考資料と根拠

すべて2026-09-18に確認した一次資料。リンク先の main やドキュメント内容は将来変わり得る。実装では採用revisionを固定する。以下の資料は仕様の背景であり、ERABIの実機性能を示す資料ではない。

[S01] TypeSafe : System One

<https://docs.typesafe.ai/concepts/system-one>

判断モデルの公開された目的・入出力を確認するための資料。ERABIとJevの内部構造の一致や、GLiClassとの派生関係を裏付けるものではない。

[S02] Knowledgator : GLiClass公式リポジトリ

<https://github.com/Knowledgator/GLiClass>

公開実装、学習例、入力機能、コードライセンス、利用可能な構成の確認。

[S03] GLiClass論文

<https://arxiv.org/html/2508.07662v1>

候補条件付き分類、uni-encoderの考え方、評価と制約の背景。性能表をERABIの性能へ転用しない。

[S04] GLiClass Multilang Miniモデルカード

<https://huggingface.co/knowledgator/gliclass-multilang-mini>

候補チェックポイント、表記された規模・言語・ライセンス。日本語が記載言語に含まれない点を確認。

[S05] GLiClass Instruct Baseモデルカード

<https://huggingface.co/knowledgator/gliclass-instruct-base-v1.0>

必要時の第二比較候補。利用する場合はその版の設定とライセンスを再確認。

[S06] SB Intuitions : ModernBERT-Ja-130M

<https://huggingface.co/sbintuitions/modernbert-ja-130m>

日本語・英語の事前学習、132Mという構成、8,192トークン、MIT License、用途上の注意。

[S07] Hintonほか : Distilling the Knowledge in a Neural Network

<https://arxiv.org/abs/1503.02531>

教師分布を生徒へ伝える知識蒸留の背景。

[S08] Guoほか : On Calibration of Modern Neural Networks

<https://proceedings.mlr.press/v70/guo17a.html>

確率校正と温度スケーリングの背景。

[S09] Ovadiaほか : Uncertainty Under Dataset Shift

<https://arxiv.org/abs/1906.02530>

分布変化下での不確実性評価。未知タスクへの無条件な校正保証をしない根拠。

[S10] Hugging Face : Transformers Installation

<https://huggingface.co/docs/transformers/installation>

ローカル保存・オフライン利用の手順。

[S11] GLiClass : pipeline.py

<https://raw.githubusercontent.com/Knowledgator/GLiClass/main/gliclass/pipeline.py>

単一選択のSoftmax、通常出力のtop1、全候補情報、入力経路の確認。

[S12] GLiClass : data_processing.py

https://raw.githubusercontent.com/Knowledgator/GLiClass/main/gliclass/data_processing.py

hard targetとsoft値の処理経路、入力整形、候補順序等の確認。

[S13] GLiClass : model.py

<https://raw.githubusercontent.com/Knowledgator/GLiClass/main/gliclass/model.py>

logitsと学習損失の公開実装。採用版で可変候補数の扱いを確認する。

[S14] GLiClass : train.py

<https://raw.githubusercontent.com/Knowledgator/GLiClass/main/train.py>

公開学習例のデータ分割を確認。ERABIでは独立groupによる事前splitを使う。

[S15] NVIDIA : CUDA on WSL User Guide

<https://docs.nvidia.com/cuda/wsl-user-guide/index.html>

WSL環境でのCUDAとドライバーの取り扱い。

[S16] OpenAI : AGENTS.md

<https://developers.openai.com/codex/guides/agents-md>

プロジェクト指示の読込と優先関係。確認時はOpenAIの学習資料へリダイレクト。

[S17] FastAPI : Deployment Concepts

<https://fastapi.tiangolo.com/deployment/concepts/>

複数プロセスでのメモリ複製など、常駐モデルの運用上の背景。

[S18] Google Antigravity : Rules

<https://antigravity.google/docs/rules-workflows/>

workspace ruleの保存先と適用方式。エディター版による差は手元で確認。

[S19] Hugging Face : ModernBERT

https://huggingface.co/docs/transformers/model_doc/modernbert

Transformers側のモデル対応とAPI。実機の対応版を固定する。

[S20] JGLUE公式リポジトリ

<https://github.com/yahoojapan/JGLUE>

日本語評価データの候補。データ別の利用条件と既存splitを確認する。

最終方針：先に大量データを作らず、先に巨大な基盤を作らず、一問の実推論から始める。正しく測れる小さな実装を作り、測定で必要になったものだけを追加する。